Number Systems I

Contemporary Digital Electronic Systems — Chapter 2

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Decimal and Binary
3. Bit Groups
4. Octal and Hexadecimal
5. Binary Codes
6. Summary

Why This Chapter Matters

Every value a digital circuit stores, computes or transmits is a pattern of bits. Reading, writing and converting between number systems, and recognising the common binary codes, is the foundational literacy for everything that follows.

After this chapter, you can:

1. Convert integers and fractions between decimal, binary, octal and hexadecimal.
2. Count in each system and recognise place values.
3. Encode a decimal digit in BCD and read it back.
4. Explain why Gray code changes one bit at a time.
5. Encode characters in ASCII; know how Unicode differs.

© Muním Zabidi

An Opening Question

The decimal number 1000 fits in four digits.

How many bits are needed to store the same value in binary, and how many hexadecimal digits?

By the end of the chapter you should answer without a calculator.

Table of Contents

1. Why This Chapter
2. Decimal and Binary
3. Bit Groups
4. Octal and Hexadecimal
5. Binary Codes
6. Summary

- The number of symbols in a counting system is its base or radix. Decimal has radix 10; digital systems use radix 2.
- A digit's value depends on its position. For 786.3:

7	8	6	.	3
10^2	10^1	10^0	.	10^{-1}

Any base b

$$N_b = a_{n-1}b^{n-1} + \cdots + a_1b^1 + a_0b^0 + a_{-1}b^{-1} + \cdots + a_{-m}b^{-m}$$

Binary Numbers

- Each digit is a bit: 0 or 1.
- Leftmost bit: MSB (most significant). Rightmost: LSB.
- Weights are powers of two.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

n	2^n	2^{-n}
0	1	1
1	2	0.5
2	4	0.25
3	8	0.125
4	16	0.0625
5	32	0.03125

Add the weights of the bits that are 1; ignore the 0s.

Example: 1011011_2

Weights:	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Bits:	1	0	1	1	0	1	1

$$1011011_2 = 64 + 16 + 8 + 2 + 1 = 91_{10}$$

Example: 1.01_2

$$1.01_2 = 2^0 + 2^{-2} = 1 + \tfrac{1}{4} = 1.25_{10}$$

© Munim Zabidi

Decimal to Binary: Sum of Weights

Subtract the greatest power of two that fits; repeat until nothing is left.

Example: 19_{10}

1. Biggest power of 2 below 19 is 16: $19 - 16 = 3$.
2. $3 - 2 = 1$.
3. $1 - 1 = 0$.
4. So $19 = 16 + 2 + 1$.

16	8	4	2	1
1	0	0	1	1

$\Rightarrow 19_{10} = 10011_2$

Decimal to Binary: Repeated Division

		remainder
37	$\div 2$	1
18	$\div 2$	0
9	$\div 2$	1
4	$\div 2$	0
2	$\div 2$	0
1	$\div 2$	1
0		(stop)

- Divide by 2, record the remainder, repeat with the quotient until it is 0.
- The first remainder is the LSB; read upwards.
- $37_{10} = 100101_2$.
- Check: $32 + 4 + 1 = 37$.

Decimal Fractions to Binary

Integer part by repeated division; fraction part by repeated multiplication by 2.

Example: 2.25_{10}

Integer: $2 \div 2 = 1$ r 0 (LSB), $1 \div 2 = 0$ r 1 (MSB) $\Rightarrow 10_2$.

Fraction:

$$\begin{array}{lcllll} 0.25 & \times & 2 & = & 0.5 & \text{keep integer 0} \\ 0.5 & \times & 2 & = & 1.0 & \text{keep integer 1; fraction is 0, stop} \end{array}$$

Fraction is $.01_2$, so $2.25_{10} = 10.01_2$.

Table of Contents

1 Why This Chapter

2 Decimal and Binary

3 **Bit Groups**

4 Octal and Hexadecimal

5 Binary Codes

6 Summary

Common Bit Groups

Name	Bits
Bit	1
Nibble	4
Byte (octet)	8
Word	16, 32 or 64

- A word is a system's native data size (32 bits on ARM).
- n bits hold values up to $2^n - 1$.
- To store x distinct values you need $\lceil \log_2 x \rceil$ bits.

n	Largest value
4	15
8	255
16	65 535
32	4 294 967 295

Storage and Speed Prefixes

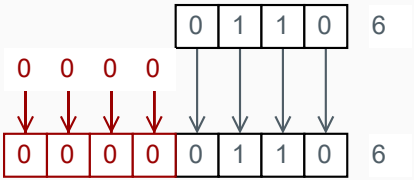
Prefix	Sym.	2^n	10^n
kilo	K	2^{10}	10^3
mega	M	2^{20}	10^6
giga	G	2^{30}	10^9
tera	T	2^{40}	10^{12}

- Storage: powers of 2. 1 KB = 1024 bytes.
- Everything else (clock, power, data rate): powers of 10. 1 Mbps = 10^6 bit/s.
- 1 MBps = 8 000 000 bit/s.

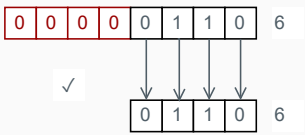
Example: addressing 8 KB

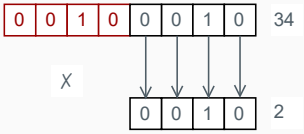
$8 \times 1024 = 8192$ bytes, so $n = \lceil \log_2 8192 \rceil = 13$ address bits.

Expanding and Shrinking Binary Numbers



Expand: pad 0s at the MSB end





Shrink: OK only if every removed bit is 0

- Truncation removes bits. If a removed upper bit is 1, the value changes.

Table of Contents

1 Why This Chapter

2 Decimal and Binary

3 Bit Groups

4 **Octal and Hexadecimal**

5 Binary Codes

6 Summary

Octal: Three Bits per Digit

- Symbols 0 to 7; each digit is exactly 3 bits.
- Conversion to and from binary is direct substitution.

0	1	2	3	4	5	6	7
000	001	010	011	100	101	110	111

1357_8 to decimal

$$1\times8^3 + 3\times8^2 + 5\times8^1 + 7\times8^0$$
$$= 512 + 192 + 40 + 7 = 751_{10}$$

2145_8 to binary

$$2\ 1\ 4\ 5 \rightarrow 010\ \ 001\ \ 100\ \ 101$$
$$= 010001100101_2$$

Decimal to Octal; Binary to Octal

2935_{10}: repeated division by 8

2935	$\div 8$	366	r	7 (LSD)
366	$\div 8$	45	r	6
45	$\div 8$	5	r	5
5	$\div 8$	0	r	5 (MSD)

$2935_{10} = 5567_8$

1101110010_2 to octal

Group in threes from the right; pad the left with 0s:

001	101	110	010
1	5	6	2

$= 1562_8$

Hexadecimal: Four Bits per Digit

Dec	Bin	Hex	Dec	Bin	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

- 16 symbols: 0 to 9 and A to F.
- Each digit is exactly 4 bits: one nibble.
- Most digital systems work in groups of 4 or 8 bits, so hex is the engineer's shorthand.
- Weight of digit i is 16^i.

© Muním Zabidi

Hexadecimal Conversions

$C1D_{16}$ to decimal

$12 \times 256 + 1 \times 16 + 13 \times 1$
$= 3072 + 16 + 13 = 3101_{10}$

3546_{10} to hex

3546	$\div 16$	221	r	10 = A (LSD)
221	$\div 16$	13	r	13 = D
13	$\div 16$	0	r	13 = D (MSD)

$3546_{10} = DDA_{16}$

$C0DE_{16}$ to binary

C	0	D	E
1100	0000	1101	1110

$= 1100000011011110_2$

10111101010_2 to hex

Group in fours from the right, pad with 0s:

0101	1110	1010
5	E	A

$= 5EA_{16}$

Fractions in Octal and Hexadecimal

0.65625_{10} to octal: multiply by 8

0.65625	$\times 8$	5.25	keep	5 (MSD)
0.25	$\times 8$	2.0	keep	2 (LSD)

$0.65625_{10} = 0.52_8$ (exact: $0.625 + 0.03125$)

24.36_8 to decimal

$2 \times 8 + 4 + 3 \times 8^{-1} + 6 \times 8^{-2}$
$= 16 + 4 + 0.375 + 0.09375 = 20.46875_{10}$

1011.11011_2 to hex

Group outwards from the binary point; pad the fraction on the right:

1011	1101	1000
B	D	8

$= \mathrm{B.D8}_{16}$

- Most decimal fractions never terminate: stop after the digits needed.

Back to the Opening Question

1000_{10}

- $2^9 = 512 \leq 1000 < 1024 = 2^{10}$, so 10 bits: 1111101000_2.
- Group in fours: 0011 1110 1000, so 3 hex digits: $3E8_{16}$.

System	Radix	Bits per digit
Binary	2	1
Octal	8	3
Hexadecimal	16	4

Table of Contents

1 Why This Chapter

2 Decimal and Binary

3 Bit Groups

4 Octal and Hexadecimal

5 **Binary Codes**

6 Summary

Decimal-Based Codes

Reference material: these codes are outside the main syllabus, but the code converters of Chapters 7 and 8 use them.

Digit	BCD	Aiken	XS-3
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

- BCD (8421): each decimal digit in 4 bits. Patterns 1010 to 1111 are unused.
- Aiken (2421): weighted; self-complementing.
- XS-3: digit + 3; unweighted; self-complementing.
- Self-complementing: inverting all four bits gives the nine's complement ($3 \leftrightarrow 6$).

641_{10} in BCD

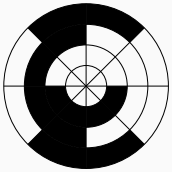
6 4 1 → `0110 0100 0001`

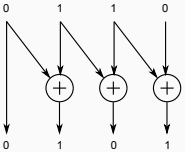
Gray Code

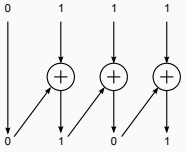
Dec	Binary	Gray
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

- Successive values differ in one bit only.
- Designed for position encoders: at a boundary, binary can flicker through wrong values; Gray is off by at most one step.
- Binary to Gray: $g_i = b_i \oplus b_{i+1}$ (with $b_n = 0$).
- Gray to binary: start at the MSB, $b_{n-1} = g_{n-1}$, then $b_i = b_{i+1} \oplus g_i$.

Gray Code Conversion







0110_2 to Gray

$g_0 = 0 \oplus 1 = 1, g_1 = 1 \oplus 1 = 0,$
$g_2 = 1 \oplus 0 = 1, g_3 = 0$
$\Rightarrow 0101_{\text{Gray}}$

0110_{Gray} to binary

$b_3 = 0, b_2 = 0 \oplus 1 = 1,$
$b_1 = 1 \oplus 1 = 0, b_0 = 0 \oplus 0 = 0$
$\Rightarrow 0100_2$

ASCII and Unicode

ASCII

- 7-bit code, 128 characters.
- First 32 codes are control codes; the rest are printable.

Example: ‘C’ is column 100, row 0011, so $1000011_2 = 43_{16}$.

Unicode

- Covers Latin, Cyrillic, Arabic, Chinese, Japanese and emoji.
- Compatible with ASCII for the first 128 code points.
- Typically encoded as UTF-8.

Error Detection: Parity

- Even parity: total number of 1s is even.
- Parity bit $p = b_3 \oplus b_2 \oplus b_1 \oplus b_0$.
- A single flipped bit gives odd parity: error detected.
- An even number of flipped bits is not detected.
- Memory with SECDED can also *correct* single-bit errors.

Char	ASCII	With even parity
S	0110101	00110101
a	1100001	11100001
l	1101100	01101100
m	1101101	11101101
!	0100001	00100001

The unused MSB of each byte carries the parity bit.

Parity Example: ASCII K

- K $= 1001011_2$ has four 1s: already even.
- Parity bit $= 0$; transmit `0 1001011`.
- One flipped bit: odd count, error detected.
- Two flipped bits: even count, error missed.
- Parity says *that* something failed, not *which* bit, so it cannot correct.

Case	Received	1s	Decision
No error	`0 1001011`	4	Accept
One bit flipped	`0 1011011`	5	Reject
Two bits flipped	`0 1011111`	6	Accept (missed)

Table of Contents

1 Why This Chapter

2 Decimal and Binary

3 Bit Groups

4 Octal and Hexadecimal

5 Binary Codes

6 **Summary**

Summary

- Every number system is positional: the weight of digit i is r^i.
- Binary is native to hardware; octal and hex group 3 or 4 bits per character.
- Decimal to binary: repeated division by 2 (integers), repeated multiplication by 2 (fractions).
- BCD stores each decimal digit in 4 bits: easy for humans, wasteful and needs special arithmetic.
- Gray code changes one bit between consecutive values: rotary encoders, asynchronous crossings.
- ASCII is the 7-bit baseline; Unicode (usually UTF-8) extends it.
- A parity bit detects any odd number of bit errors.

Looking ahead

Chapter 3 extends these ideas to *signed* numbers (sign–magnitude, one's and two's complement) and binary arithmetic.